```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.tree import DecisionTreeClassifier,plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
df = pd.read_csv("/content/TrafficDataset.csv")

print("Original Dataset :")
print(df.head())
```

```
Original Dataset :
          Time        Date Day of the week  CarCount  BikeCount  BusCount  \
0  12:00:00 AM  10-10-2023         Tuesday        13          2         2
1  12:15:00 AM  10-10-2023         Tuesday        14          1         1
2  12:30:00 AM  10-10-2023         Tuesday        10          2         2
3  12:45:00 AM  10-10-2023         Tuesday        10          2         2
4   1:00:00 AM  10-10-2023         Tuesday        11          2         1

   TruckCount  Total Traffic Situation
0          24     41            normal
1          36     52            normal
2          32     46            normal
3          36     50            normal
4          34     48            normal
```

```
df['Hour'] = pd.to_datetime(df['Time']).dt.hour
df
```

```
/tmp/ipykernel_694/3424054523.py:1: UserWarning: Could not infer format, so each element will be parsed individually, fallir
  df['Hour'] = pd.to_datetime(df['Time']).dt.hour
```

	Time	Date	Day of the week	CarCount	BikeCount	BusCount	TruckCount	Total	Traffic Situation	Hour
0	12:00:00 AM	10-10-2023	Tuesday	13	2	2	24	41	normal	0
1	12:15:00 AM	10-10-2023	Tuesday	14	1	1	36	52	normal	0
2	12:30:00 AM	10-10-2023	Tuesday	10	2	2	32	46	normal	0
3	12:45:00 AM	10-10-2023	Tuesday	10	2	2	36	50	normal	0
4	1:00:00 AM	10-10-2023	Tuesday	11	2	1	34	48	normal	1
...	...	...	...	...	...	...	...	...	...	...
2971	10:45:00 PM	09-11-2023	Thursday	6	0	2	34	42	normal	22
2972	11:00:00 PM	09-11-2023	Thursday	5	0	2	24	31	normal	23
2973	11:15:00 PM	09-11-2023	Thursday	11	2	2	32	47	normal	23
2974	11:30:00 PM	09-11-2023	Thursday	5	2	2	37	46	normal	23
2975	11:45:00 PM	09-11-2023	Thursday	10	1	0	25	36	normal	23

2976 rows × 10 columns

```
df = df.drop(columns=['Time','Date'])
```

```
df
```

	Day of the week	CarCount	BikeCount	BusCount	TruckCount	Total	Traffic Situation	Hour
0	Tuesday	13	2	2	24	41	normal	0
1	Tuesday	14	1	1	36	52	normal	0
2	Tuesday	10	2	2	32	46	normal	0
3	Tuesday	10	2	2	36	50	normal	0
4	Tuesday	11	2	1	34	48	normal	1
...	...	...	...	...	...	...	...	...
2971	Thursday	6	0	2	34	42	normal	22
2972	Thursday	5	0	2	24	31	normal	23
2973	Thursday	11	2	2	32	47	normal	23
2974	Thursday	5	2	2	37	46	normal	23
2975	Thursday	10	1	0	25	36	normal	23

2976 rows × 8 columns

```
df['Traffic Situation'] = df['Traffic Situation'].replace({
    'Low' : 'Normal',
    'Normal' : 'Normal',
    'High' : 'Heavy',
    'Heavy' : 'Heavy'
})
```

```
df.head(20)
```

	Day of the week	CarCount	BikeCount	BusCount	TruckCount	Total	Traffic Situation	Hour
0	Tuesday	13	2	2	24	41	normal	0
1	Tuesday	14	1	1	36	52	normal	0
2	Tuesday	10	2	2	32	46	normal	0
3	Tuesday	10	2	2	36	50	normal	0
4	Tuesday	11	2	1	34	48	normal	1
5	Tuesday	15	1	1	39	56	normal	1
6	Tuesday	14	2	2	27	45	normal	1
7	Tuesday	13	2	1	20	36	normal	1
8	Tuesday	7	0	0	26	33	normal	2
9	Tuesday	13	0	0	34	47	normal	2
10	Tuesday	15	2	0	38	55	normal	2
11	Tuesday	5	2	0	37	44	normal	2
12	Tuesday	9	1	0	34	44	normal	3
13	Tuesday	8	0	2	35	45	normal	3
14	Tuesday	7	0	0	34	41	normal	3
15	Tuesday	10	1	2	38	51	normal	3
16	Tuesday	82	7	3	10	102	low	4
17	Tuesday	71	3	0	30	104	normal	4
18	Tuesday	89	10	2	10	111	low	4
19	Tuesday	77	3	0	18	98	low	4

```
le = LabelEncoder()

df['Day of the week'] = le.fit_transform(df['Day of the week'])
df['Traffic Situation'] = le.fit_transform(df['Traffic Situation'])

df.head(20)
```

	Day of the week	CarCount	BikeCount	BusCount	TruckCount	Total	Traffic Situation	Hour
0	5	13	2	2	24	41	3	0
1	5	14	1	1	36	52	3	0
2	5	10	2	2	32	46	3	0
3	5	10	2	2	36	50	3	0
4	5	11	2	1	34	48	3	1
5	5	15	1	1	39	56	3	1
6	5	14	2	2	27	45	3	1
7	5	13	2	1	20	36	3	1
8	5	7	0	0	26	33	3	2
9	5	13	0	0	34	47	3	2
10	5	15	2	0	38	55	3	2
11	5	5	2	0	37	44	3	2
12	5	9	1	0	34	44	3	3
13	5	8	0	2	35	45	3	3
14	5	7	0	0	34	41	3	3
15	5	10	1	2	38	51	3	3
16	5	82	7	3	10	102	2	4
17	5	71	3	0	30	104	3	4
18	5	89	10	2	10	111	2	4
19	5	77	3	0	18	98	2	4

```
x = df.drop(columns=['Traffic Situation'])
y = df['Traffic Situation']
```

```
selector = SelectKBest(score_func=f_classif, k=4)
x_selected= selector.fit_transform(x, y)
selected_features = x.columns[selector.get_support()]

print("\nSelected Features :")
print(selected_features)
```

```
Selected Features :
Index(['CarCount', 'BikeCount', 'BusCount', 'Total'], dtype='object')
```

```
x_selected = pd.DataFrame(
    x_selected,
    columns=selected_features)
```

```
x_train,x_test,y_train,y_test = train_test_split(
    x_selected,y,test_size=0.2,random_state=42)
```

```
model = DecisionTreeClassifier(
    criterion = 'gini',
    random_state = 42
)
model.fit(x_train,y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

```
root_index = model.tree_.feature[0]
root_node = x_selected.columns[root_index]

print("\nRoot Node :")
print(root_node)
```

```
Root Node :
Total
```

```
y_pred = model.predict(x_test)
```

```
accuracy = accuracy_score(y_test,y_pred)

print("\nAccuracy :")
print(accuracy)
```

```
Accuracy :
0.9295302013422819
```

```
print("\nConfusion Matrix :")
print(confusion_matrix(y_test,y_pred))
```

```
Confusion Matrix :
[[ 86   0   0   0]
 [  0  32   0   7]
 [  0   0  41  12]
 [  0   5  18 395]]
```

```
new_traffic = x_selected.iloc[[0]]
prediction = model.predict(new_traffic)

if prediction[0]==1:
  print("\nNew traffic prediction : Normal")
else:
  print("\nNew traffic prediction : Heavy")
```

```
New traffic prediction : Heavy
```

```
plt.figure(figsize=(20,10))

plot_tree(
    model,
    feature_names = selected_features,
    class_names =[str(c) for c in model.classes_],
    filled = True,
    max_depth=3
)

plt.show()
```

